FORLER: Federated Offline Reinforcement Learning with Q-Ensemble and Actor Rectification

1st Nan Qiao
Central South University.
School of Computer Science
Changsha, China
214707006@csu.edu.cn

2nd Sheng Yue
School of Cyber Science and Technology
Sun Yat-sen University
Shenzhen, China
yuesh5@mail.sysu.edu.cn

***Abstract*—In Internet-of-Things (IoT) systems, federated learning has advanced online reinforcement learning (RL) by enabling parallel policy training without sharing raw data. However, interacting with real environments online can be risky and costly, motivating offline federated RL (FRL), where local devices learn from fixed datasets. Despite its promise, offline FRL may break down under low-quality, heterogeneous data. Offline RL tends to get stuck in local optima, and in FRL one device's suboptimal policy can degrade the aggregated model, i.e., policy pollution. We present FORLER, combining Q-ensemble aggregation on the server with actor rectification on devices. The server robustly merges device Q-functions to curb policy pollution and shift heavy computation off resource-constrained hardware without compromising privacy. Locally, actor rectification enriches policy gradients via a zeroth-order search for high-Q actions plus a bespoke regularizer that nudges the policy toward them. A δ-periodic strategy further reduces local compute. We theoretically provide safe policy improvement performance guarantees. Extensive experiments show that FORLER consistently outperforms strong baselines under varying data quality and heterogeneity.**

***Index Terms*—Edge Computing, Federated Learning, Offline Reinforcement Learning**

I. Introduction

Sequential decision-making is a fundamental capability underpinning Internet of Things applications, from autonomous driving to multi-robot control [1], [2]. Recent progress has been driven by deep reinforcement learning [3], with a major shift from single- to multi-agent DRL that improves exploration [4]. A persistent challenge, however, is the need to transmit substantial data to a central controller, stressing bandwidth and risking privacy [5]. This has motivated federated reinforcement learning (FRL) [6]. In FRL, local devices interact with the environment online, update policies locally, and periodically aggregate them on a server [5]. However, such online interaction is often impractical, costly, or unsafe in real systems. For example, training autonomous-driving policies on public roads poses substantial safety risks [7]. Offline FRL mitigates these issues by learning from fixed local datasets with offline RL methods [8]–[12], Yet, it remains challenging at both the local update and global aggregation stages.

At the local update stage, the value function guides policy improvement, but the objective is non-concave, so policy-gradient methods often get stuck in local optima, especially with low-quality data [13]. To stabilize offline training, many works add behavior cloning [8], [10], which implicitly assumes behavior actions are near-optimal [9]. When data quality is uneven, coverage limited, or the BC weight large, behavior cloning tethers the policy to the behavior policy, dampens Q-driven improvement, and under federated aggregation increases the risk of suboptimal performance and pollution.

Furthermore, current global aggregation methods have notable limitations, as our analysis shows. As shown in Fig. 2, the performance of the advanced method degrades markedly when devices with low-quality datasets join training. These devices learn policies that are locally optimal to their data but pollute the global policy during aggregation. We term this *policy pollution*. Moreover, many offline FRL methods deliberately push more computation onto devices to preserve privacy and reduce uplink traffic [8], [11]. This design burdens resource-constrained IoT hardware, increasing latency and energy costs and worsening instability when compute capabilities are heterogeneous. The contaminated global policy then harms other devices, degrading otherwise strong local policies.

To address these issues, we propose federated offline reinforcement learning with Q-ensemble and actor rectification (FORLER), featuring two components: Q-ensemble aggregation and actor rectification. During server-side aggregation, we collect Q-parameters from all devices and run a new offline RL algorithm based on Q-ensembles, which both improves offline RL itself and robustly aggregates cross-device information to produce updated Q-functions and policies. On devices, actor rectification exploits conservative value functions by augmenting standard policy gradients with a zeroth-order search for high-Q actions; a tailored regularizer then pulls the policy toward these actions. We also adopt a "δ-periodic" strategy that samples only every τ updates to reduce local computation.

In a nutshell, our main contributions are summarized below.

- We design an ensemble-based offline RL algorithm for server-side aggregation that mitigates policy pollution and better accommodates statistical heterogeneity.
- We introduce actor rectification for local training, alleviating local optima. The "δ-periodic" strategy further improves computational efficiency.
- We prove that FORLER enjoys the safe policy improvement guarantee and demonstrate consistent gains over prior methods through extensive experiments.

II. Preliminaries

This section reviews federated learning and offline reinforcement learning, and then presents the system model.

Federated learning trains models locally on devices and aggregates their updates on a central server without sharing raw data. In FedAvg, the global objective is $F(w) = \sum_{k=1}^{\mathcal{K}} p_k F_k(w)$, with dataset-size weights $p_k = \frac{|\mathcal{D}_k|}{|\mathcal{D}|}, |\mathcal{D}| = \sum_{k=1}^{\mathcal{K}} |\mathcal{D}_k|$. However, size-based weighting may misrepresent data quality, and simple linear aggregation can perform poorly under heterogeneous data and system conditions.

Reinforcement learning can be formalized as an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \mu_0, \gamma)$ with goal $\max_\pi \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right]$. Q-learning estimates $Q^*(\mathbf{s}, \mathbf{a})$; actor–critic methods update a policy (actor network) using a value estimator (critic network) by minimizing the Bellman error $(Q(\mathbf{s}, \mathbf{a}) - \mathcal{B}^\pi Q(\mathbf{s}, \mathbf{a}))^2$ with $\mathcal{B}^\pi Q(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\mathbf{s}' \sim T(\cdot|\mathbf{s},\mathbf{a})}\left[r(\mathbf{s}, \mathbf{a}) + \gamma\, \mathbb{E}_{\mathbf{a}' \sim \pi(\cdot|\mathbf{s}')} Q_\phi(\mathbf{s}', \mathbf{a}')\right]$. In offline RL, learning minimizes this error over a fixed dataset, while the policy maximizes the critic's predicted return. TD3BC encourages staying on the data manifold [10]:

$$\pi_{td3bc} \leftarrow \arg\max_{\pi}\ \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim\mathcal{D}}\left[\lambda Q(\mathbf{s}, \pi(\mathbf{s})) - (\pi(\mathbf{s}) - \mathbf{a})^2\right]. \tag{1}$$

where λ trades off value maximization and behavior cloning. Alternatively, CQL learns conservative value estimates [14]:

$$\begin{aligned} Q_{cql}^{t+1} \leftarrow \arg\min_{Q}\ & \mathbb{E}_{\mathbf{s},\mathbf{a},s'\sim\mathcal{D}}\left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi Q^t(\mathbf{s}, \mathbf{a})\right)^2\right] \\ & + \omega\left(\mathbb{E}_{s\sim\mathcal{D},\ a\sim\mu(\cdot|\mathbf{s})}[Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s},\mathbf{a}\sim\mathcal{D}}[Q(\mathbf{s}, \mathbf{a})]\right), \end{aligned} \tag{2}$$

where $\hat{\mathcal{B}}^\pi Q = r(\mathbf{s}, \mathbf{a}) + \gamma\ \mathbb{E}_{\mathbf{a}' \sim \pi(\cdot|\mathbf{s}')}\left[\min_{n=1,2} Q_n(\mathbf{s}', \mathbf{a}')\right]$ and ω is a positive coefficient. Such regularization curbs overestimation yet can be sensitive to the number of critics.

We illustrate the system model in Fig. 1. Each of the $\mathcal{K}$ devices and the central server holds an offline dataset. Device k stores $\mathcal{D}_k = \{\mathbf{s}_j, \mathbf{a}_j, r_j, \mathbf{s}'_j\}_{j=1}^{|\mathcal{D}_k|}$, and the server stores $\mathcal{D}_o = \{\mathbf{s}_j, \mathbf{a}_j, r_j, \mathbf{s}'_j\}_{j=1}^{|\mathcal{D}_o|}$. Devices run offline RL with actor-side correction locally and send parameters to the server. The server performs offline RL with a Q-ensemble to update global parameters and then broadcasts them back to devices.

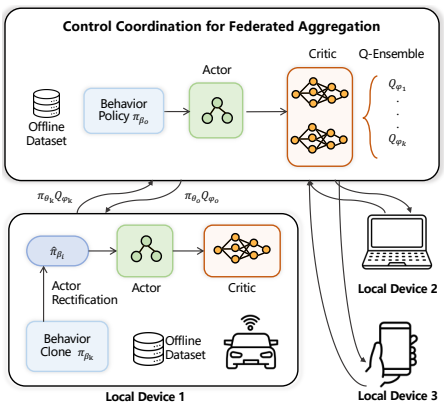


Fig. 1. The architecture of FORLER. The system integrates a suite of devices coordinated by a server. Both the individual devices and the server are equipped with offline datasets and compute independently.

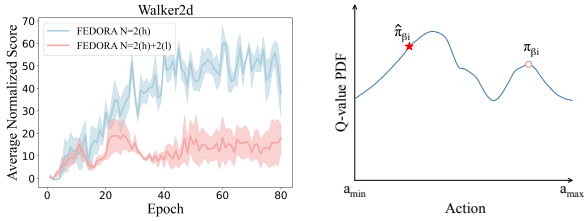


Fig. 2. The performance of advanced algorithms, *e.g.* FEDORA. (Left) Overall performance difference between high-quality datasets and mixed (high-quality + additional low-quality) datasets. (Right) A visual representation of the Q-value vs. Action. The circle symbolizes the predicted action derived from the updating policy of FEDORA, while the pentacle represents the corresponding action predicted by the updating policy of our method.

III. Proposed Method

In this section, we first introduce a motivating example to show that advanced algorithms suffer from policy pollution and become trapped in local optima. We then introduce FORLER, which couples Q ensemble aggregation on the server with δ periodic actor rectification for local updates. This dual design addresses these shortcomings and yields a more robust framework for offline FRL.

A. The Motivating Example

We conducted a series of experiments using the walker2d dataset to assess the impact of data quality on learning performance. Specifically, we utilized two devices, each operating on medium-expert data with a dataset size $|\mathcal{D}_k| = 20k$. Our findings, illustrated in the left part of Fig 2, reveal a counter-intuitive result: despite the extra dataset added in the second experiment, its performance was significantly lower compared to the first. These results highlight policy pollution and its impact, with the extra low-quality data degrading the learned policy and overall performance.

Additionally, we identify a key limitation of advanced methods: critic-only regularization does not enable devices to learn effective coordination policies. The right panel of Fig 2 shows the Q value landscape at a given training step. The circle denotes the action produced by a representative advanced method under its current update, while the pentacle denotes our approach. The former becomes trapped in a poor local optimum. As a result, devices fail to leverage the conservative value function globally, leading to suboptimal policies.

B. Local Updates with δ-periodic Actor Rectification

a) Local policy evaluation: Obviously, extrapolation error is a tricky problem in local offline policy evaluation. Because training does not interact with the environment online, the state–action distribution induced by the current policy diverges from the dataset distribution [10], [12], [14]. We therefore adopt CQL to mitigate this shift and curb overestimation on out-of-distribution actions, which is as follows:

$$Q_k^{\tau+1} \leftarrow \arg\min_{Q} \frac{1}{2}\mathbb{E}_{\mathbf{s},\mathbf{a},s'\sim\mathcal{D}_k}\left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^{\pi_k} Q_k^k(\mathbf{s}, \mathbf{a})\right)^2\right]$$

$$+\omega_c \left(\mathbb{E}_{s\sim\mathcal{D}_k, a\sim\mu(\cdot|s)}[Q(\mathbf{s},\mathbf{a})] - \mathbb{E}_{\mathbf{s},\mathbf{a}\sim\mathcal{D}_k}[Q(\mathbf{s},\mathbf{a})]\right), \quad (3)$$

which is similar with Eq. (3) and has been fully covered.

b) Local policy improvement with δ-periodic Actor Rectification: In light of the challenges in the local optima problem, we adopt the following policy-improvement process:

$$\pi_k^{\tau+1} \leftarrow \arg\max_{\pi} \mathbb{E}_{s\sim\mathcal{D}_k, a\sim\pi(\cdot|s)}\left[\hat{Q}_k(\mathbf{s},\mathbf{a})\right] - \alpha_1\left(\pi - \hat{\pi}_{\beta_k}^{\tau}\right)^2 - \alpha_2\left(\pi - \pi_0\right)^2, \quad (4)$$

where $\hat{\pi}_{\beta_k}^{\tau}$ represents the action proposed by a zeroth-order optimizer, with α_1 and α_2 acting as positive coefficients.

Notably, $(\pi - a)^2$ have been utilized in TD3BC and FEDORA [9], [10], where a is the 'seen' action in the dataset. Nevertheless, we replace it with $\hat{\pi}_{\beta_k}^{\tau}$, which is the 'optimized' action and makes our method perform well even if the quality of the dataset is low. Specifically, in the training process, we initialize a Gaussian distribution $\mathcal{N}(x_k, y_k)$. Over I iterations, generate a population of N individual actions $\hat{\mathcal{A}}_k = \{\hat{\mathbf{a}}_k^n\}_{n=1}^N$ sampled from the distribution $\mathcal{N}(x_k, y_k)$. For each action in the population, it estimates Q-values using the critic Q_k^{τ}. Subsequently, the parameters x_k and y_k of the Gaussian distribution are updated according to a predefined update equation:

$$x_k^{i+1} = \frac{\sum_{k=1}^{K}\exp(\beta Q_k^n)\hat{\mathbf{a}}_k^n}{\sum_{m=1}^{N}\exp(\beta Q_k^m)}, \quad y_k^{i+1} = \sqrt{\sum_{k=1}^{K}\left(\hat{\mathbf{a}}_k^n - x_k^j\right)^2}. \quad (5)$$

That is, we select the optimal candidate action $\hat{\pi}_{\beta_k}^{\tau+1}$ based on the highest Q-value among the sampled actions and the current policy. Nevertheless, the described method poses significant challenges due to its *high computational complexity and extensive sampling* requirements. These aspects render it less suitable for environments with constrained computing and storage capabilities, particularly for devices.

To address this limitation, we introduce an alternative approach, which we refer to as *δ-periodic Actor Rectification*. This new strategy involves computing $\hat{\pi}_{\beta_k}^{\tau}$ at regular intervals, each time δ elapses. Accordingly, $\hat{\pi}_{\beta_k}^{\tau}$ is updated as

$$\hat{\pi}_{\beta_k}^{\tau+1} \doteq \begin{cases} \arg\max_{\hat{\pi}_{\beta_k}\in\hat{\mathcal{A}}_k\cup\pi_k(\mathbf{s})} Q_k^{\tau}(\mathbf{s},\hat{\pi}_{\beta_k}(\mathbf{s})), & \tau\in\Gamma^{\delta}, \\ \arg\max_{\hat{\pi}_{\beta_k}\in\hat{\pi}_{\beta_k}^{\tau}\cup\pi_k(\mathbf{s})} Q_k^{\tau}(\mathbf{s},\hat{\pi}_{\beta_k}(\mathbf{s})), & \text{otherwise}, \end{cases}$$

where $\Gamma^{\delta} \doteq \{0, \delta, 2\delta, \cdots\}$. The method avoids computing $\hat{\pi}_{\beta_k}^{\tau}$ at every iteration, thereby simplifying the procedure and substantially reducing computation and sampling overhead. Moreover, beyond these practical benefits, we have also established through theoretical analysis that this method enhances the security policy, ensuring a more robust and efficient performance. The complete pseudo code of *δ-periodic Actor Rectification* is shown in Algorithm 1.

C. Server Aggregation with Q-Ensemble

We design a server side Q ensemble for aggregation on the server dataset $\mathcal{D}_o$. Each device contributes two critic heads, hence $2K$ in total. Inspired by SAC-N in offline RL [12], the server builds a pessimistic target with the pointwise minimum

Algorithm 1: δ-periodic Actor Rectification

Input: Critic Q_k^{τ}, $\mathbf{s}$, $\hat{\pi}_{\beta_k}^{\tau}$, and τ from Algorithm 2

if $\tau \in \Gamma^{\delta}$ **then**
  Initialize $\mathcal{N}(x_k, y_k)$
  for *iteration* $i = 1$ *to* I **do**
    Construct $\hat{\mathcal{A}}_k$ by sampling $\hat{\mathbf{a}}_k^n \sim \mathcal{N}(x_k, y_k)$ for $n = 1, \ldots, N$.
    Estimate Q-values for K individuals in the population $\{Q_k^1(\mathbf{s}, \hat{\mathbf{a}}_k^n)\}_{n=1}^N$
    Update x_k and y_k according to Eq. (5)
  end
  Obtain the picked candidate action $\hat{\pi}_{\beta_k}^{\tau+1} = \arg\max_{\hat{\pi}_{\beta_k}\in\hat{\mathcal{A}}_k\cup\pi_k(\mathbf{s})} Q_k^{\tau}(\mathbf{s}, \hat{\pi}_{\beta_k}(\mathbf{s}))$
else
  $\hat{\pi}_{\beta_k}^{\tau+1} = \arg\max_{\hat{\pi}_{\beta_k}\in\hat{\pi}_{\beta_k}^{\tau}\cup\pi_k(\mathbf{s})} Q_k^{\tau}(\mathbf{s}, \hat{\pi}_{\beta_k}(\mathbf{s}))$
end

Output: $\hat{\pi}_{\beta_k}^{\tau+1}$

and updates the iterates π_o^t and Q_o^t to curb policy pollution and overestimation. Accordingly, the critic is updated by

$$Q_o^{t+1} \leftarrow \arg\min_{Q} \mathbb{E}_{\mathbf{s},\mathbf{a},\mathbf{s}'\sim\mathcal{D}_o}\left[\left(Q(\mathbf{s},\mathbf{a}) - \widehat{\mathcal{B}}_K^{\pi_o}Q(\mathbf{s},\mathbf{a})\right)^2\right] + \omega^s\left(\mathbb{E}_{\mathbf{s}\sim\mathcal{D}_o,\ \mathbf{a}'\sim\pi_o(\cdot|\mathbf{s})}Q(\mathbf{s},\mathbf{a}') - \mathbb{E}_{\mathbf{s},\mathbf{a}\sim\mathcal{D}_o}Q(\mathbf{s},\mathbf{a})\right), \quad (6)$$

with empirical Bellman operator $\widehat{\mathcal{B}}_K^{\pi_o}Q(\mathbf{s},\mathbf{a}) = r(\mathbf{s},\mathbf{a}) + \gamma \cdot \mathbb{E}_{\mathbf{a}'\sim\pi_o(\cdot|\mathbf{s}')}\left[\min_{i=1,\ldots,2K} Q_i(\mathbf{s}',\mathbf{a}') - \beta\log\pi_o(\mathbf{a}'\mid\mathbf{s}')\right]$, where $\omega^s \geq 0$ weights a conservative behavior regularizer and $\beta \geq 0$ is the entropy temperature. For the policy improvement, we maximize the corresponding pessimistic surrogate:

$$\pi_o^{t+1} \leftarrow \arg\max_{\pi} \mathbb{E}_{\mathbf{s}\sim\mathcal{D}_o,\ \mathbf{a}\sim\pi(\cdot|\mathbf{s})}\left[\widehat{\Phi}_K^{\pi_o}Q(\mathbf{s},\mathbf{a})\right], \quad (7)$$

where $\widehat{\Phi}_K^{\pi_o}Q(\mathbf{s},\mathbf{a}) = \min_{i=1,\ldots,2K} Q_i(\mathbf{s},\mathbf{a}) - \beta\log\pi(\mathbf{a}\mid\mathbf{s})$. The server alternates (6) and (7) each round and broadcasts the updated π_o^{t+1} and Q_o^{t+1} to devices.

D. FORLER

Outlined in Algorithm 2, we design the FORLER algorithm for offline FRL, which follows a structured process integrating both server and device-side computations. On each device, at iteration τ, a minibatch of transitions is sampled from the local dataset $\mathcal{D}_k$. The actor-rectification procedure (Algorithm 1) then updates the local policy π_k and critic Q_k using the sampled data and the current rectified action $\hat{\pi}_{\beta_k}^{\tau}$. After the local step, each device returns its updated critic Q_k^{Γ} to the server. The server aggregates the received critics $\{Q_k^{\Gamma}\}_{k=1}^{\mathcal{K}}$ to obtain the global critic Q_o^{t+1} and policy π_o^{t+1} according to the aggregation equations, and broadcasts them to all devices. This process repeats until convergence, yielding the final global policy π_o and a set of optimized local policies $\{\pi_k\}_{k=1}^{\mathcal{K}}$.

IV. Theoretical Analysis

In this section, we provide performance guarantees for FEDORA and establish its safe policy improvement property.

Algorithm 2: FORLER

Input: α_1, α_2 , $\mathcal{D}_o$, $\{\mathcal{D}_k\}_{k=1}^{\mathcal{K}}$

Server initializes π_o^0 and Q_o^0 and sends agents;
for $t = 0$ **to** T **do**
  for $k = 1$ **to** K **do**
    `// Device side`
    Receive Q_o^t and π_o^t from server
    Set $Q_k^0 = Q_o^t$ and $\pi_k^0 = \pi_o^t$
    for $\tau = 0$ **to** Γ **do**
      Sample a random minibatch of $\mathbf{s}$ samples $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')$ from $\mathcal{B}_k$
      Apply Algorithm 1 with critic Q_k^τ, $\mathbf{s}$, $\hat{\pi}_{\beta_k}^\tau$, and τ to obtain $\hat{\pi}_{\beta_k}^{\tau+1}$
      Calculate critic $Q_k^{\tau+1}$ according to Eq. (3)
      Calculate policy $\pi_k^{\tau+1}$ according to Eq. (4)
    end
    device k sends updated Q_k^Γ to the server
  end
  `// Server side`
  Receive all local $\{Q_k^\Gamma\}_{k=1}^{\mathcal{K}}$
  Calculate critic Q_o^{t+1} according to Eq. (6)
  Calculate policy π_o^{t+1} according to Eq. (7)
  Distribute Q_o^{t+1} and π_o^{t+1} to all devices
end

Output: Global policy π_o and local policies $\{\pi_k\}_{k=1}^{\mathcal{K}}$

To begin, we extend the lower-bound result of the CQL theorem to account for sampling error. Following prior work, we assume that the reward function and the transition dynamics satisfy standard concentration properties [8], [14]. For any $(\mathbf{s}, \mathbf{a}) \in \mathcal{D}$, the following holds with probability at least $1-\delta$:

Assumption 1. *For all $(\mathbf{s}, \mathbf{a}) \in \mathcal{D}$, the reward r and the transition kernel T satisfy, with probability $\geq 1-\delta$,*

$$\left| r - r(\mathbf{s}, \mathbf{a}) \right| \leq \frac{C_{r,\delta}}{\sqrt{|\mathcal{D}(\mathbf{s}, \mathbf{a})|}}, \quad \left\| \hat{T}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) - T(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \right\|_1 \leq \frac{C_{T,\delta}}{\sqrt{|\mathcal{D}(\mathbf{s}, \mathbf{a})|}}.$$

$|\mathcal{D}(\mathbf{s}, \mathbf{a})|$ denote the number of occurrences of $(\mathbf{s}, \mathbf{a})$ in $\mathcal{D}$, let $\hat{T}$ be the empirical transition kernel, and let $\|\cdot\|_1$ be the L_1 distance. Assume $|r(\mathbf{s}, \mathbf{a})| \leq R_{\max}$, which implies $\|\hat{Q}^k\|_\infty \leq R_{\max}/(1-\gamma)$. Using Assumption 1, we can bound the discrepancy between the empirical and true Bellman operators for any policy π:

$$\begin{aligned} &\left| (\hat{\mathcal{B}}^\pi \hat{Q}^k) - (\mathcal{B}^\pi \hat{Q}^k) \right| \\ &\leq \left| r - r(\mathbf{s}, \mathbf{a}) \right| + \gamma \left| \sum_{\mathbf{s}'} \left(\hat{T}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) - T(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \right) \mathbb{E}_{\mathbf{a}' \sim \pi} \left[\hat{Q}^k(\mathbf{s}', \mathbf{a}') \right] \right| \\ &\leq \frac{C_{r,\delta} + \gamma\, C_{T,\delta} \frac{2R_{\max}}{1-\gamma}}{\sqrt{|\mathcal{D}(\mathbf{s}, \mathbf{a})|}} = \frac{C_{r,T,\delta}}{\sqrt{|\mathcal{D}(\mathbf{s}, \mathbf{a})|}}. \end{aligned} \tag{8}$$

This bound quantifies the potential overestimation induced by sampling error. The constant $C_{r,T,\delta}$ is a function of $C_{r,\delta}$ and $C_{T,\delta}$, and it inherits a $\sqrt{\log(1/\delta)}$ dependence. The analysis follows prior work that measures sampling effects via the empirical Bellman operator. Based on these assumptions, we obtain the following safe policy improvement theorem.

Theorem 1 (Safe policy improvement)**.** *Let π_k^* be the policy obtained by optimizing Eq. (4). Define*

$$D\left(\pi_k, \hat{\pi}_{\beta_k}\right)(\mathbf{s}) \triangleq \frac{1 - \hat{\pi}_{\beta_k}(\pi_k(\mathbf{s}) \mid \mathbf{s})}{\hat{\pi}_{\beta_k}(\pi_k(\mathbf{s}) \mid \mathbf{s})},$$

and let $d^{\pi_k}(\mathbf{s})$ denote the marginal discounted state-visitation distribution of policy π_k. Then, for $\alpha > 0$ and $\tau \in (0, 1)$,

$$\begin{aligned} J(\pi_k^*) - J(\hat{\pi}_{\beta_k}) \geq\ & \frac{\alpha}{1-\gamma} \mathbb{E}_{\mathbf{s} \sim d^{\pi_k^*}} \left[D\left(\pi_k^*, \hat{\pi}_{\beta_k}\right)(\mathbf{s}) \right] \\ &+ \frac{\tau}{1-\tau} \mathbb{E}_{\mathbf{s} \sim d^{\pi_k^*}} \left[\left(\pi_k^*(\mathbf{s}) - \hat{\pi}_{\beta_k}(\mathbf{s}) \right)^2 \right] \\ &- \frac{\tau}{1-\tau} \mathbb{E}_{\substack{\mathbf{s} \sim d^{\hat{\pi}_{\beta_k}} \\ \mathbf{a} \sim \hat{\pi}_{\beta_k}(\cdot|\mathbf{s})}} \left[\left(a_k - \hat{\pi}_{\beta_k}(\mathbf{s}) \right)^2 \right]. \end{aligned} \tag{9}$$

The inequality shows that under the stated conditions FEDORA achieves safe improvement over the behavior policy. The first term captures the benefit from deviating from the behavior policy, and the last two terms correspond to a quadratic penalty for the target policy and a baseline term under the behavior policy. The complete proof is following:

Proof. For FORLER, we have the following iterative update:

$$\begin{aligned} &\arg\min_{Q_k} \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q_k(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^{\pi_k} \hat{Q}_k^t(\mathbf{s}, \mathbf{a}) \right)^2 \right] \\ &+ \alpha \mathbb{E}_{\mathbf{s} \sim \mathcal{D}_k} \left[\mathbb{E}_{\mathbf{a} \sim \tilde{\pi}_k} \left[Q_k(\mathbf{s}, \mathbf{a}) \right] - \mathbb{E}_{\mathbf{a} \sim \hat{\pi}_{\beta_k}} \left[Q_k(\mathbf{s}, \mathbf{a}) \right] \right], \end{aligned} \tag{10}$$

where $\tilde{\pi}_k(\mathbf{a}|\mathbf{s}) = 1$ if and only if $a_k = \pi_k(\mathbf{s})$. Let $\hat{Q}_k^{t+1}$ be the fixed point of solving Equation (10) by setting the derivative of Eq. (10) with respect to Q_k to be 0, then we have that

$$\hat{Q}_k^{t+1}(\mathbf{s}, \mathbf{a}) = \hat{\mathcal{B}}^{\pi_k} \hat{\mathbf{Q}}_\mathbf{k}^\mathbf{t}(\mathbf{s}, \mathbf{a}) - \alpha \left(\frac{\mathbf{I}_{\mathbf{a_k} = \pi_\mathbf{k}(\mathbf{s})}}{\hat{\pi}_{\beta_\mathbf{k}}(\mathbf{a_k}|\mathbf{s})} - \mathbf{1} \right), \tag{11}$$

where I is the indicator function. Denote $D(\pi_k, \hat{\pi}_{\beta_k})(\mathbf{s}) = \frac{1}{\hat{\pi}_{\beta_k}(\pi_k(\mathbf{s})|\mathbf{s})} - 1$, and we obtain the difference between the value function $\hat{V}_k(\mathbf{s})$ and the original value function as:

$$\hat{V}_k(\mathbf{s}) = V_k(\mathbf{s}) - \alpha D(\pi_k, \hat{\pi}_{\beta_k})(\mathbf{s}), \tag{12}$$

Then, the policy that minimizes the loss function defined in Eq. (4) is equivalently obtained by maximizing

$$\begin{aligned} (1-\tau) \left(J(\pi_k) - \alpha \frac{1}{1-\gamma} \mathbb{E}_{\mathbf{s} \sim d_{\hat{M}_k}^{\pi_k}(\mathbf{s})} \left[D(\pi_k, \hat{\pi}_{\beta_k})(\mathbf{s}) \right] \right) \\ - \tau \mathbb{E}_{\mathbf{s} \sim d_{\hat{M}_k}^{\pi_k}(\mathbf{s})} \left[(\pi_k(\mathbf{s}) - \hat{\pi}_{\beta_k})^2 \right]. \end{aligned} \tag{13}$$

Therefore, we obtain that

$$\begin{aligned} &(1-\tau) \left(J(\pi_k^*) - \alpha \frac{1}{1-\gamma} \mathbb{E}_{\mathbf{s} \sim d_{\hat{M}_k}^{\pi_k^*}(\mathbf{s})} \left[D(\pi_k^*, \hat{\pi}_{\beta_k})(\mathbf{s}) \right] \right) \\ &- \tau \mathbb{E}_{\mathbf{s} \sim d_{\hat{M}_k}^{\pi_k^*}(\mathbf{s})} \left[(\pi_k^*(\mathbf{s}) - \hat{\pi}_{\beta_k})^2 \right] \\ &\geq (1-\tau) J(\hat{\pi}_{\beta_k}) - \tau \mathbb{E}_{\mathbf{s} \sim d_{\hat{M}_k}^{\hat{\pi}_{\beta_k}}(\mathbf{s}), \mathbf{a} \sim \hat{\pi}_{\beta_k}(\mathbf{a}|\mathbf{s})} \left[(a_k - \hat{\pi}_{\beta_k})^2 \right]. \end{aligned} \tag{14}$$

Then, from Eq. (14) we obtain the result. □

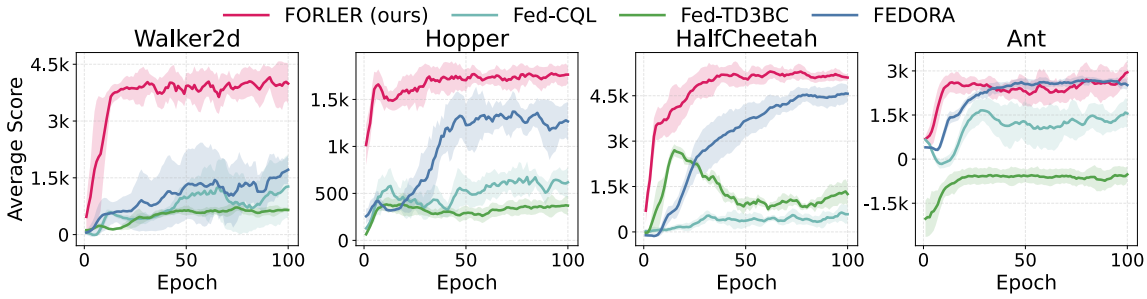


Fig. 3. The performance of FEDORA against baselines.

Fig. 4. Performance comparison of different methods on policy pollution problem.

V. Simulation

In this section, we evaluate our proposed algorithm by answering the following questions:

1) How does the overall performance of our method compare to baselines?
2) What is the effect of the number of devices, the size of the dataset, and critical hyperparameters?
3) Is Algorithm FORLER effective in addressing the problem of policy pollution?
4) What is the impact of the δ-periodic strategy?

A. Experimental Setup

We evaluate the benefits of our method (FORLER) using datasets from the widely recognized offline RL benchmark D4RL, which is built upon the MuJoCo simulation environment [15]. This benchmark encompasses 4 complex continuous control tasks (HalfCheetah, Walker2d, Ant, and Hopper). Additionally, we compare FORLER with several baselines:

- *Conservative Q-Learning (CQL)* , a wildly-studied offline RL algorithm, is executed in a centralized way [16]. We re-run CQL using the code of OfflineRL-Kit [17].
- *Federated Conservative Q-Learning (Fed-CQL)*. It works by changing the CQL rules locally on the devices with a set number of gradient steps and then combining the parameters in the FedAvg way at the server.
- *Federated TD3 with Behavior Cloning (Fed-TD3BC)*, a combination of FedAvg and TD3BC [10], operates by performing TD3BC updates locally on the devices with a certain number of gradient steps and subsequently aggregating the policy parameters at the server.
- *FEDORA* is the advanced offline FRL algorithm [9]. We reproduce FEDORA following their paper and the open-source implementations.

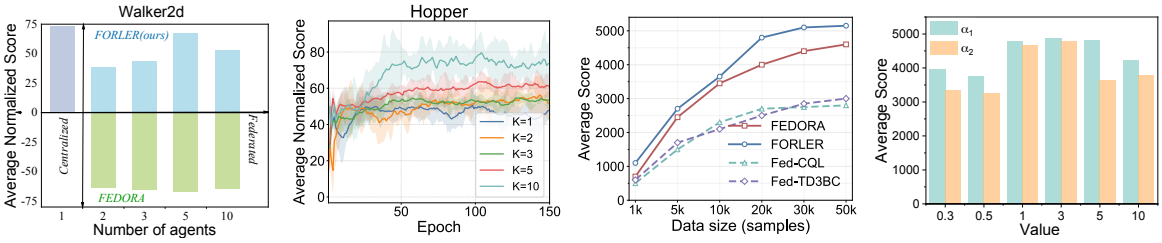
We implement FORLER in PyTorch 2.1.2 and run all experiments on Ubuntu 20.04.4 LTS with four NVIDIA GeForce RTX 3090 GPUs. Unless noted otherwise, all metrics are averaged over 3 independent runs with different random seeds.

B. Experimental Evaluation

As shown in Fig. 3, we compare FORLER with Fed-TD3BC, Fed-CQL, and FEDORA on D4RL MuJoCo tasks—Walker2d, Hopper, HalfCheetah, and Ant. FORLER achieves higher scores and faster convergence across all four tasks, consistently outperforming the baselines. The average-score curves indicate rapid, stable improvement and strong final policies, supporting FORLER as a robust and efficient solution for offline federated RL.

To assess the effects of device count, dataset size, and key hyperparameters on FORLER, we ran ablations summarized in Figs. 5. In Fig. 5(a), FORLER consistently outperforms FEDORA across agent counts, remaining robust as devices increase. In Fig. 5(b), FORLER maintains a steady advantage, while FEDORA varies more with the number of samples N. Furthermore, Fig. 5(c) shows FORLER scales well, achieving higher average scores and exploiting larger datasets more effectively. As shown in Fig. 5(d), hyperparameter sweeps over α_1 and α_2 yield stable performance, indicating low sensitivity and minimal tuning needs. Overall, FORLER delivers superior, scalable performance with reduced dependence on device number, data volume, and hyperparameters.

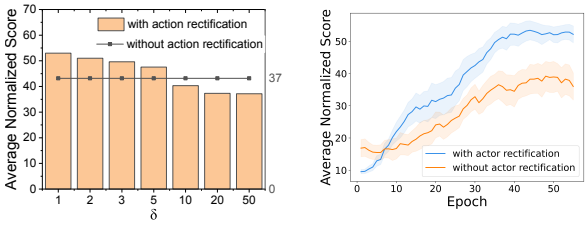
To investigate whether FORLER mitigates policy pollution, we follow Fig. 4 and form a mixed-quality setup with six devices: four high-quality devices using expert, medium-expert, medium, and medium-replay data, and two low-quality devices using random data. We examine whether injecting random data degrades either high-quality devices or server-side aggregation. Across all four high-quality regimes, FORLER remains robust



(a) Performance on devices (b) Impact of number of devices. (c) The impact of data size (d) The impact of hyperparameters

Fig. 5. Analyzing the influence of device count, dataset size, and key hyperparameter.

on both device and server sides, outperforming FEDORA and FedCQL and showing little to no degradation. High-quality devices retain strong scores despite random peers, and the aggregated policy is only minimally affected. Overall, FORLER effectively prevents policy pollution, preserving high-quality policy and fitting federated settings with uneven data quality.



(a) Impact of value δ (b) Ablation study

Fig. 6. Actor rectification improves performance: δ sensitivity and ablation.

In an ablation study depicted in Figure 6, we examine the efficacy of actor rectification and the impact of the δ-periodic strategy on the performance of our FORLER algorithm. As shown in Fig. 6(a), we observe that implementing actor rectification consistently yields higher scores across all δ values, underscoring its positive effect on the learning process. Moreover, the performance remains relatively stable across various δ settings, suggesting that while the actor rectification contributes to performance gains, the specific choice of δ within a certain range does not significantly alter the outcomes. Fig. 6(b) further reinforces the benefits of actor rectification, where the learning curve with rectification outpaces that without, indicating not only improved performance but also an accelerated rate of learning when actor rectification is applied.

VI. Conclusion

In this study, we introduce FORLER, a novel offline FRL approach for Internet of Things applications that tackles policy pollution and local optima. FORLER combines server side Q ensemble aggregation with δ-periodic actor rectification to strengthen global policy robustness and improve local update efficiency in heterogeneous data settings. Extensive experiments demonstrate that FORLER significantly surpasses existing methods in performance, proving its effectiveness in handling diverse datasets and mitigating the risks of suboptimal policies. This research marks a substantial step forward in the application of offline FRL, offering promising solutions for complex, real-world decision-making scenarios.

References

[1] D. Zhang, J. Liang, K. Guo, S. Lu, Q. Wang, R. Xiong, Z. Miao, and Y. Wang, "Carplanner: Consistent auto-regressive trajectory planning for large-scale reinforcement learning in autonomous driving," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 17 239–17 248.

[2] N. Qiao, S. Yue, Y. Zhang, and J. Ren, "Popec: Paoi-centric task offloading with priority over unreliable channels," *arXiv preprint arXiv:2303.15119*, 2023.

[3] L. WANG, X. WU, Y. WANG, Z. XIAO, L. LI, and A. FEI, "On uav serving node deployment for temporary coverage in forest environment: A hierarchical deep reinforcement learning approach," *Chinese Journal of Electronics*, vol. 32, no. 4, pp. 760–772, 2023.

[4] A. M. Seid, G. O. Boateng, B. Mareri, G. Sun, and W. Jiang, "Multi-agent drl for task offloading and resource allocation in multi-uav enabled iot edge network," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4531–4547, 2021.

[5] J. Qi, Q. Zhou, L. Lei, and K. Zheng, "Federated reinforcement learning: Techniques, applications, and open challenges," *arXiv preprint arXiv:2108.11887*, 2021.

[6] H. Jin, Y. Peng, W. Yang, S. Wang, and Z. Zhang, "Federated reinforcement learning with environment heterogeneity," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2022.

[7] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *arXiv preprint arXiv:2005.01643*, 2020.

[8] S. Yue, Y. Deng, G. Wang, J. Ren, and Y. Zhang, "Federated offline reinforcement learning with proximal policy evaluation," *Chinese Journal of Electronics*, vol. 33, no. 6, pp. 1360–1372, 2024.

[9] D. Rengarajan, N. Ragothaman, D. Kalathil, and S. Shakkottai, "Federated ensemble-directed offline reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 37, pp. 6154–6179, 2024.

[10] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," *Advances in neural information processing systems*, vol. 34, pp. 20 132–20 145, 2021.

[11] N. Qiao, S. Yue, J. Ren, and Y. Zhang, "Fova: Offline federated reinforcement learning with mixed-quality data." [Online]. Available: https://drive.google.com/file/d/1R_LEjqPaj8OkEZ-U5pPCfQulQpoGTPd7/view?pli=1

[12] G. An, S. Moon, J.-H. Kim, and H. O. Song, "Uncertainty-based offline reinforcement learning with diversified q-ensemble," *Advances in neural information processing systems*, vol. 34, pp. 7436–7447, 2021.

[13] L. Pan, L. Huang, T. Ma, and H. Xu, "Plan better amid conservatism: Offline multi-agent reinforcement learning with actor rectification," in *International conference on machine learning*. PMLR, 2022, pp. 17 221–17 237.

[14] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1179–1191, 2020.

[15] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4rl: Datasets for deep data-driven reinforcement learning," *arXiv preprint arXiv:2004.07219*, 2020.

[16] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1179–1191, 2020.

[17] Y. Sun, "Offlinerl-kit: An elegant pytorch offline reinforcement learning library," https://github.com/yihaosun1124/OfflineRL-Kit, 2023.